AAT REPORT ON

Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL

COURSE: ADVANCE DIGITAL LOGIC DESIGN

(23EC5PE1AD)

Submitted by

Chandana V **1BM25EC408**

Submitted to

Prof. Ajaykumar Devarapalli Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
BMS College of Engineering Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

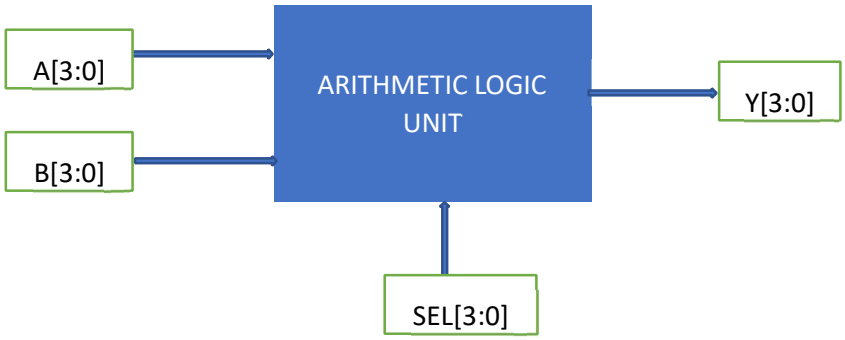
2026 – 2027

VERILOG CODES

1. ARITHMETIC LOGIC UNIT (ALU)

An ALU is a combinational circuit that performs arithmetic and logical operations on binary data. It performs operations such as addition, subtraction, AND, OR, and XOR. The operation is selected using control or select lines. The output depends on the input data and the selected operation.

DESIGN



VERILOG CODE

```
module alu (     input
[3:0] A, B,     input
[2:0] sel,     output reg
[3:0] Y

);
always @(*) begin
case (sel)
        3'b000: Y = A + B;
        3'b001: Y = A - B;
        3'b010: Y = A & B;
        3'b011: Y = A | B;
3'b100: Y = A ^ B;
default: Y = 4'b0000;
endcase end  endmodule
```

Test bench: module tb alu; reg [3:0] A,

B; reg [2:0] sel; wire [3:0] Y; alu uut

(.A(A), .B(B), .sel(sel), .Y(Y)); initial

begin A=4'd5; B=4'd3; sel=3'b000;

#10; sel=3'b001; #10; sel=3'b010;

#10; sel=3'b011; #10; sel=3'b100;

#10;

$finish; end

endmodule

2. COMPARATOR

A comparator is a combinational circuit used to compare two binary numbers. It determines whether the first number is greater than, less than, or equal to the second number. It produces three outputs: greater than, less than, and equal. The outputs depend on the values of the input numbers.

DESIGN



VERILOG CODE

module comparator (

input [3:0] A, B, output

gt, lt, eq

);

assign gt = (A > B); assign

lt = (A < B); assign eq =

(A == B); endmodule

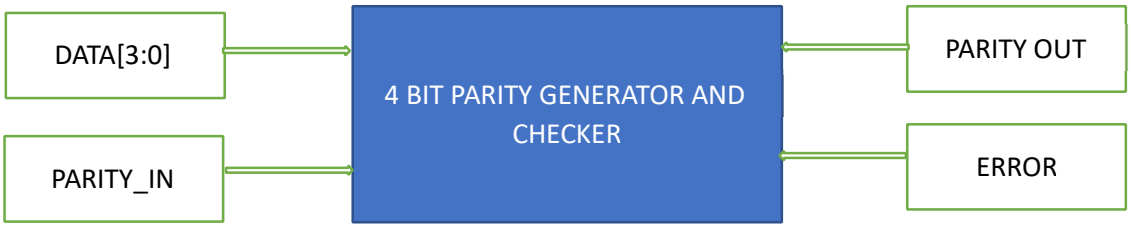
TEST BENCH:

```
module tb_comparator; reg [3:0] A, B; wire gt, lt, eq;
comparator uut (.A(A), .B(B), .gt(gt), .lt(lt), .eq(eq));
initial begin     A=5; B=3; #10;
   A=2; B=7; #10;
   A=4; B=4; #10;
   $finish; end
endmodule
```

3. 4-BIT PARITY GENERATOR AND CHECKER

A parity generator generates a parity bit to detect errors in binary data. It uses XOR gates to generate even or odd parity for a 4-bit input. A parity checker checks the received data along with its parity bit. If the parity is incorrect, an error is detected.

DESIGN



VERILOG CODE

```
module parity (     input
[3:0]  data,          input
parity_in,          output
parity_out,         output
error
```

);

assign parity_out = ^data; assign

error = ^{data, parity_in};

endmodule

TEST BENCH:

module tb_parity; reg [3:0] data; reg parity_in; wire parity_out, error; parity uut

(.data(data), .parity_in(parity_in), .parity_out(parity_out), .error(error)); initial

begin data=4'b1010;

parity_in=1'b0; #10;

parity_in=1'b1; #10; data=4'b1110;

parity_in=1'b0; #10; $finish; end endmodule

4. 4-BIT BARALLEL SHIFTER

A barrel shifter is a combinational circuit used to shift binary data by a specified number of positions. It can perform left or right shifts depending on the control input. The shift amount is selected using control lines. It performs shifting in a single operation without requiring multiple clock cycles.

DESIGN



VERILOG CODE

module barrel_shifter (

```
input [3:0] A,     input
[1:0] shamt,     input dir,
output reg [3:0] Y

);
always @(*) begin
if (dir == 1'b0)        Y
= A << shamt;     else

      Y = A >> shamt;
end  endmodule
```

TEST BENCH:

```
module tb_barrel; reg [3:0] A; reg [1:0] shamt; reg dir; wire
[3:0] Y;  barrel_shifter uut (.A(A), .shamt(shamt), .dir(dir),
.Y(Y)); initial begin

   A=4'b1010; shamt=1; dir=0; #10;
shamt=2; dir=1; #10;     shamt=0; dir=0;
#10;

   $finish; end
endmodule
```

5. TIMER MODULE

A timer module counts clock pulses to measure a specific time interval. It uses a counter that increments with each active clock edge. When the counter reaches the specified value, the timer generates an output signal. A reset signal initializes the counter to zero.

DESIGN



VERILOG CODE

```
module timer (
   input clk, reset,
output reg done

);
reg [3:0] count;  always @(posedge clk or
posedge reset) begin
if (reset) begin        count <= 0;        done <=
0;     end     else if (count == 9) begin
count <= 0;        done <= 1;     end     else begin
count <= count + 1;        done <=

0;     end end endmodule
```

TEST BENCH:

```
module tb_timer;
reg clk, reset; wire done;  timer uut (.clk(clk),

.reset(reset), .done(done));

always #5 clk = ~clk;
initial begin
clk=0; reset=1;
#10 reset=0;     #120
```

$finish; end

endmodule

6. RING COUNTER

A ring counter is a sequential circuit made using a shift register with its output connected back to its input. It circulates a single logic 1 through the register during each clock pulse. A reset initializes the counter to a known state. For a 4-bit ring counter, the sequence normally contains four distinct states.

DESIGN



VERILOG CODE

```
module ring_counter (
input clk, reset,
output reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0001;     else
      q <= {q[2:0], q[3]};
end endmodule
```

TEST BENCH: module tb_ring; reg clk, reset;

wire [3:0] q; ring_counter uut (.clk(clk),

.reset(reset), .q(q));

always #5 clk = ~clk; initial begin clk=0;

reset=1; #10 reset=0; #80 $finish; end

endmodule

7. JOHNSON COUNTER

A Johnson counter is a sequential circuit in which the inverted output of the last flip-flop is connected to the input of the first flip-flop. register shifts its data on every active clock edge. An n-bit Johnson counter can generate up to 2n distinct states. It is used in sequence generation and digital control circuits.

DESIGN



VERILOG CODE

```
module johnson_counter (
   input clk, reset,    output
reg [3:0] q

);
always @(posedge clk or posedge reset) begin
   if (reset)       q <=
4'b0000;    else       q <=
{q[2:0], ~q[3]}; end endmodule
```

TEST BENCH

```
module tb_johnson; reg clk, reset; wire [3:0] q;
johnson_counter uut (.clk(clk), .reset(reset), .q(q));
```

always #5 clk = ~clk; initial begin clk=0;

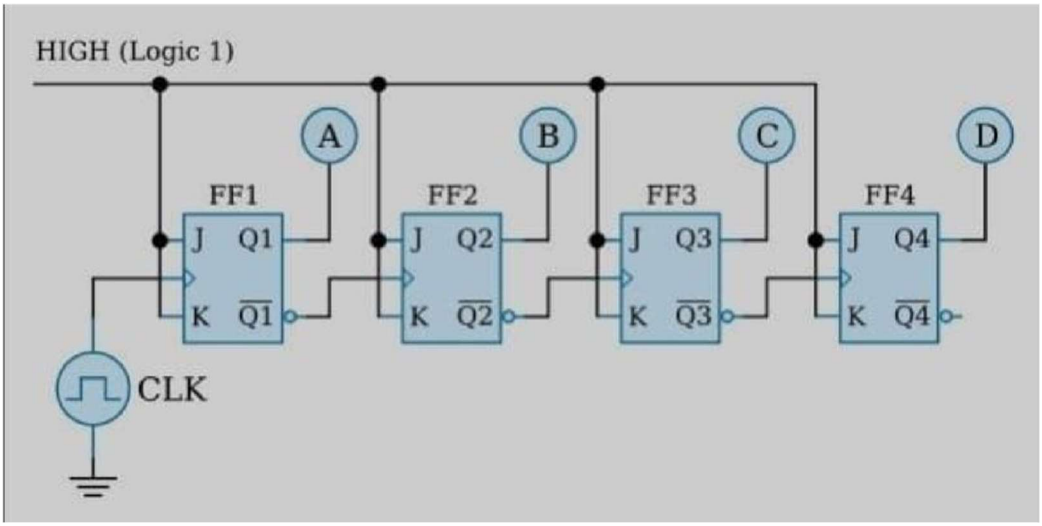
reset=1; #10 reset=0; #100 $finish; end

endmodule

8. SYNCHRONOUS COUNTER

In a synchronous counter, all flip-flops receive the same clock signal. All flip-flops change their states together at the active clock edge. It provides faster operation because the flip-flops are triggered simultaneously.

DESIGN



VERILOG CODE

module sync_counter (

input clk, reset,

output reg [3:0] q

);

always @(posedge clk or posedge reset) begin

```
if (reset)        q <= 4'b0000;     else
q <= q + 1'b1; end endmodule
```

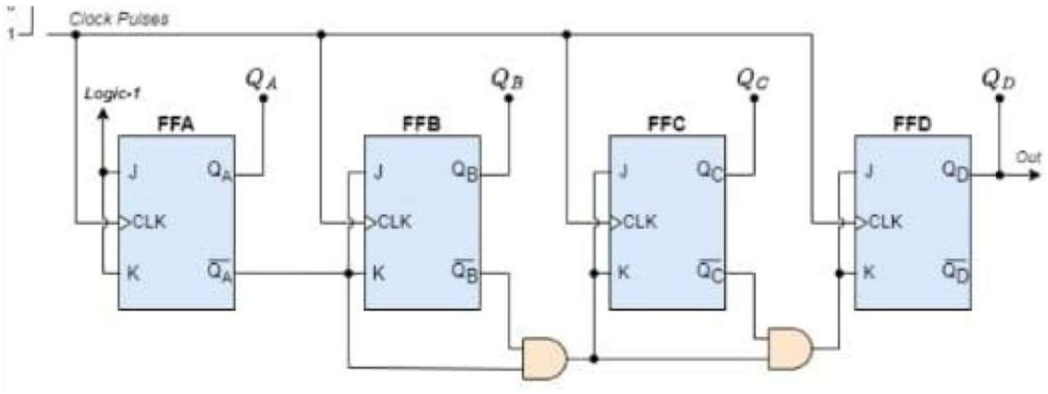
TEST BENCH:

```
module tb_sync;
reg clk, reset; wire
[3:0] q;  sync_counter uut (.clk(clk),
.reset(reset), .q(q));
always #5 clk = ~clk; initial begin     clk=0;
reset=1;     #10 reset=0;     #180 $finish; end
endmodule
```

9. ASYNCHRONOUS COUNTER

In an asynchronous counter, only the first flip-flop receives the external clock directly. The output of one flip-flop acts as the clock for the next flip-flop. The changes propagate through the counter, causing cumulative propagation delay.

DESIGN



VERILOG CODE

```
module async_counter (
   input clk, reset,
output reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset) q[0] <= 0;    else      q[0] <= ~q[0]; end
always @(negedge q[0] or posedge reset) begin    if
(reset) q[1] <= 0;    else      q[1] <= ~q[1]; end
always @(negedge q[1] or posedge reset) begin    if
(reset) q[2] <= 0;    else      q[2] <= ~q[2]; end
always @(negedge q[2] or posedge reset) begin    if
(reset) q[3] <= 0;    else      q[3] <= ~q[3]; end
endmodule
```

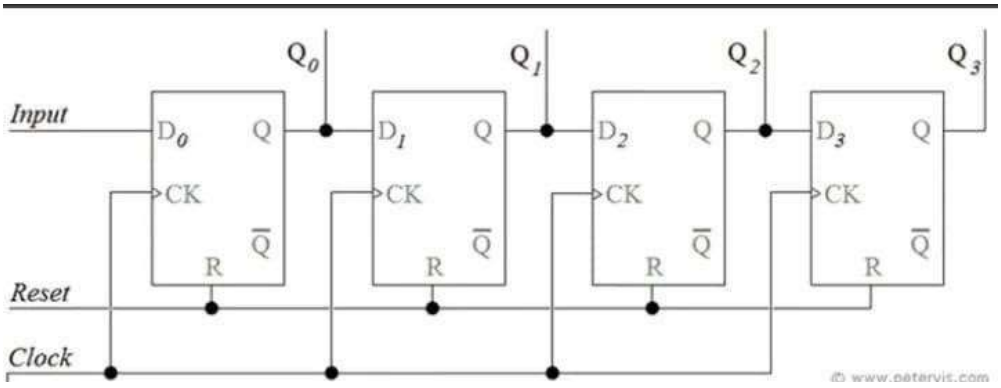
TEST BENCH:

```
module tb_async; reg
clk, reset; wire [3:0]
q; async_counter uut
(.clk(clk),
.reset(reset), .q(q));
always #5 clk =
~clk; initial begin
clk=0; reset=1;
#10 reset=0;
#180 $finish; end
endmodule
```

10. SHIFT REGISTER

A shift register is a sequential circuit used to store and shift binary data. It consists of a group of flip-flops connected in series. At every active clock edge, data shifts from one flip-flop to the next. It is used for temporary data storage and serial-to-parallel or parallel-to-serial conversion.

DESIGN



VERILOG CODE

```
module shift_register (
input clk, reset, serial_in,     output
reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0000;     else

q <= {q[2:0], serial_in};

end endmodule
```

TEST BENCH:

```
module tb_shift; reg
```

```
clk, reset, serial_in; wire
[3:0] q;  shift_register
uut ( .clk(clk),
.reset(reset),
.serial_in(serial_in),
.q(q));

always #5 clk = ~clk; initial begin

clk=0; reset=1; serial_in=0;

   #10 reset=0;
serial_in=1; #10;
serial_in=0; #10;
serial_in=1; #10;
serial_in=1; #10;     #30
$finish; end endmodule
```

11. DIVIDE BY N COUNTER

A divide-by-N counter divides the input clock frequency by a specified number N. It counts clock pulses from zero to N−1 and then resets to zero. A suitable output signal is generated based on the counter state. It is used in clock division and frequency control circuits.

DESIGN



VERILOG CODE

```
module divide_by_n (
   input clk, reset,
output reg clk_out

);
reg [3:0] count;  always @(posedge clk or
posedge reset) begin
if (reset) begin        count <= 0;        clk_out
<= 0;     end     else if (count == 9) begin
count <= 0;         clk_out <= ~clk_out;     end
else begin         count <= count + 1'b1;     end
end
endmodule
```

TEST BENCH:

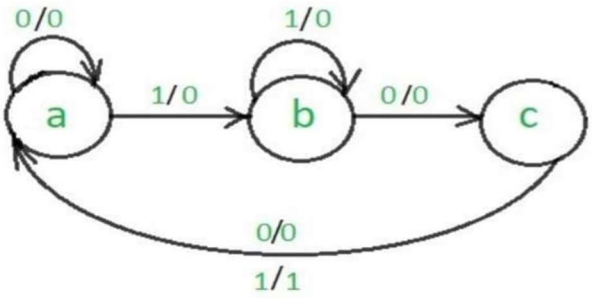
```
module tb_divide;
reg clk, reset; wire clk_out;  divide_by_n uut (.clk(clk),
.reset(reset), .clk_out(clk_out));
always #5 clk = ~clk; initial begin     clk=0; reset=1;     #10 reset=0;
#250 $finish; end endmodule
```

12. MELAY FSM

A Mealy finite state machine produces its output based on the present state and current input. Its output can change immediately when the input changes. It generally requires fewer states than an equivalent Moore machine. It is used in sequence detectors and digital control systems.

This example detects the input sequence 101. The output becomes 1 when the final 1 completes the sequence, and the output depends on both the current state and input.

DESIGN



VERILOG CODE

```
module mealy_fsm (
   input clk, reset, din,
output reg detected

); reg [1:0] state; local param S0=0, S1=1,
S2=2; always @(posedge clk or posedge reset)
begin     if (reset) begin         state <= S0;
detected <= 0;     end     else begin
detected <= 0;         case (state)            S0: if
(din) state <= S1;                else state <= S0;
S1: if (din) state <= S1;                else state <=
S2;                    S2: if (din) begin
state <= S1;                    detected <= 1;
end                else state <= S0;  default: state
<= S0;        endcase
end end  endmodule
```

TEST BENCH

```
module tb_mealy; reg
clk, reset, din; wire detected;
mealy_fsm uut (.clk(clk),
```

```
.reset(reset), .din(din),
.detected(detected));

always #5 clk = ~clk; initial begin     clk=0; reset=1; din=0;

   #10 reset=0;     din=1;
#10;     din=0; #10;
din=1; #10;     din=0;
#10;     din=1;
#10;     din=0; #10;

   $finish;

end
endmodule
```

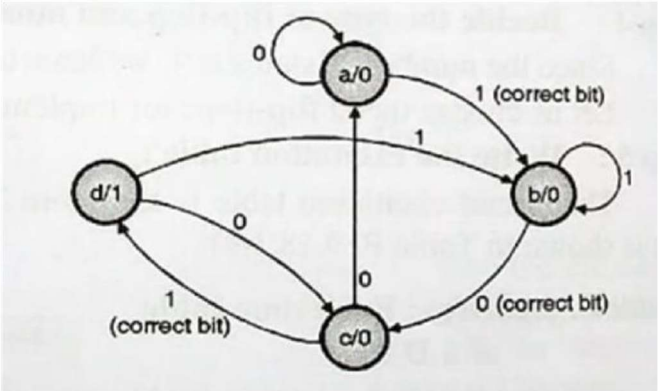
13. MOORE FSM

A Moore finite state machine produces its output based only on the present state. Its output changes when the machine transitions to another state. The output is generally more stable because it does not depend directly on the current input. It is used in controllers and sequence detection circuits.

This example also detects the sequence 101. The output becomes 1 in a dedicated detection state, so it depends only on the current state

.

DESIGN



VERILOG CODE

```
module moore_fsm (
   input clk, reset, din,     output
detected

); reg [1:0] state;

localparam S0=0, S1=1, S2=2, S3=3; assign
detected = (state == S3); always @(posedge
clk or posedge reset) begin     if (reset)
state <= S0;     else begin         case (state)

          S0: if (din) state <= S1;
else state <= S0;           S1: if
(din) state <= S1;                else
state <= S2;          S2: if (din)
state <= S3;               else state
<= S0;
        S3: if (din) state <= S1;
else state <= S2;
default: state <= S0;
     endcas
e      end
end
endmodule
```

TEST BENCH: module

```
tb_moore;  reg clk,
reset, din; wire
```

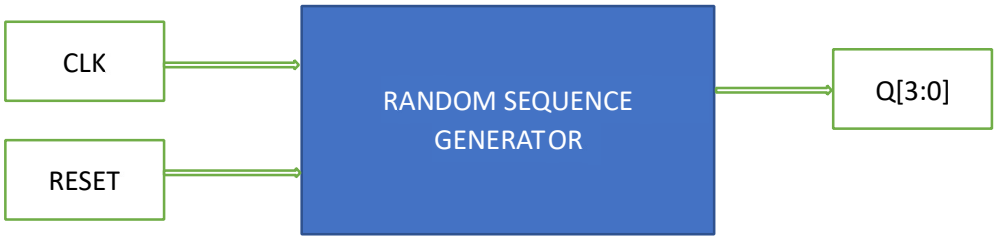
```
detected;  moore_fsm uut (.clk(clk), .reset(reset), .din(din),
.detected(detected)); always #5 clk = ~clk; initial begin     clk=0;
reset=1; din=0;     #10 reset=0;     din=1; #10;     din=0; #10;
din=1; #10;     din=0; #10;     din=1; #10;

   #20 $finish;
end  endmodule
```

14. RANDOM SEQUENCE GENERATOR

A random sequence generator produces a sequence of binary values that appears random. It can be implemented using a Linear Feedback Shift Register (LFSR). The circuit uses shift registers and XOR feedback to generate different bit patterns. It is commonly used in testing digital circuits and generating pseudorandom sequences.

DESIGN



VERILOG CODE

```
module lfsr (     input
clk, reset,     output
reg [3:0] q

);
```

```
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0001;     else          q <=
{q[2:0], q[3] ^ q[2]}; end  endmodule
```

TEST BENCH:

```
module tb_lfsr;  reg clk, reset; wire
[3:0] q; lfsr uut (.clk(clk),
.reset(reset), .q(q)); always #5 clk =
~clk; initial begin     clk=0; reset=1;
#10 reset=0;     #200
$finish;
end
endmodule
```